© 2026 ANSYS, Inc. or affiliated companies
Unauthorized use, distribution, or duplication prohibited.

ansys-pyetk

ANSYS, Inc.
Southpointe
2600 Ansys Drive
Canonsburg, PA 15317
ansysinfo@ansys.com
http://www.ansys.com
(T) 724-746-3304
(F) 724-514-9494

Jan 30, 2026

ANSYS, Inc. and ANSYS Europe, Ltd. are UL registered ISO 9001:2015 companies.

CONTENTS

1 Getting started **3**
- 1.1 Installation . . . 3
- 1.2 Architecture . . . 6

2 User Interface **9**

3 API reference **11**

4 Examples **13**

5 Release notes **15**
- 5.1 0.1.0 - January 30, 2026 . . . 15

6 What is this library? **17**

Python Module Index **19**

Index **21**

Useful links: *Installation* | Source repository | Issues

The Electronic Transformer ToolKit (PyETK) is a custom solution for electronic transformer design.

Getting started Learn how to install the Electronic Transformer toolKit (PyETK), understand its architecture and use this toolkit's common methods to develop electronic transformer models.

Getting started User interface How to use the user interface.

User Interface API reference Understand the APIs available for the PyETK.

API reference

CHAPTER

ONE

GETTING STARTED

This section explains how to install the Electronic Transformer toolKit (PyETK). It then explains this toolkit's architecture and provides an example of how to use its common methods.

Installation Learn how to install the PyAEDT Common Toolkit.

Installation Architecture Learn about this toolkit's architecture.

Architecture

1.1 Installation

1.1.1 Download links

The following installers are available for different operating systems:

Table 1: Available Installers

Installer Link	Operating System
Windows Installer	Windows
Ubuntu 22.04 Installer	Ubuntu 22.04
Ubuntu 24.04 Installer	Ubuntu 24.04

1.1.2 Installing the `Electronic Transformer toolKit (PyETK)`

Windows

First step is installing the `PyETK`. In order to do so, follow the next steps.

1. Download the necessary installer from the latest available release. The file should be named `PyETK-Installer-Windows.exe`.
2. Execute the installer.
3. Search for the `PyETK` and run it.

The `PyETK` window should appear at this stage.

Linux

Ubuntu

Prerequisites:

1. **OS** supported for **Ubuntu(24.04 and 22.04)**.

2. Update `apt-get` repository and install the following packages with **sudo** privileges: **wget, gnome, libffi-dev, libssl-dev, libsqlite3-dev, libxcb-xinerama0 and build-essential** packages with **sudo** privileges

```
sudo apt-get update -y
sudo apt-get install wget gnome libffi-dev libssl-dev libsqlite3-dev libxcb-
↪xinerama0 build-essential -y
```

3. Install **zlib** package

```
wget https://zlib.net/current/zlib.tar.gz
tar xvzf zlib.tar.gz
cd zlib-*
make clean
./configure
make
sudo make install
```

To install the `Template Toolkit`, follow below steps.

1. Download the necessary installer from the latest available release. The file should be named `Toolkit-Template_*.zip`.
2. Execute the below command on the terminal

```
unzip Toolkit-Template_*.zip
./installer.sh
```

3. Search for the `PyETK` and run it.

The `PyETK` window should appear at this stage.

To uninstall the `PyETK`, follow below steps.

1. Go to File menu. Click Uninstall option.
2. Click `Uninstall` button.

1.1.3 Python installation

The Toolkit Template can be installed like any other open source package.

From PyPI, you can either install both the backend and user interface (UI) methods or install only the backend methods.

To install both the backend and UI methods, run this command:

```
pip install ansys-pyetk[all]
```

If you only need the common API, install only the backend methods with this command:

```
pip install ansys-pyetk
```

1.1.4 For developers

You can be up and running with four lines of code:

```
git clone https://github.com/ansys/ansys-pyetk
cd ansys-pyetk
python -m pip install -U pip uv
```

(continues on next page)

(continued from previous page)

```
uv venv
uv pip install -e .
```

Now you can run it with:

```
uv run run_toolkit
```

Details

Installing Pytools installer in developer mode allows you to modify the source and enhance it.

Before contributing to the project, please refer to the PyAnsys Developer's guide. You need to follow these steps:

1. Start by cloning this repository:

   ```
   git clone https://github.com/ansys/ansys-pyetk
   ```

2. Create a fresh-clean Python environment and activate it. Refer to the official venv documentation if you require further information:

   ```
   # Create a virtual environment
   python -m uv venv .venv

   # Activate it in a POSIX system
   source .venv/bin/activate

   # Activate it in Windows CMD environment
   .venv\Scripts\activate.bat

   # Activate it in Windows Powershell
   .venv\Scripts\Activate.ps1
   ```

3. Make sure you have the latest version of pip:

   ```
   python -m pip install -U pip uv
   ```

4. Install the project in editable mode:

   ```
   python -m uv pip install -e .[tests,doc]
   ```

5. Finally, verify your development installation by running:

   ```
   uv run pytest tests -v
   ```

Style and testing

This project uses pre-commit. Install with:

```
uv pip install pre-commit
uv run pre-commit install
```

This now runs `pre-commit` for each commit to ensure you follow project style guidelines. For example:

```
git commit -am 'fix style'
isort....................................................................Passed
black....................................................................Passed
blacken-docs.............................................................Passed
flake8...................................................................Passed
codespell................................................................Passed
pydocstyle...............................................................Passed
check for merge conflicts................................................Passed
debug statements (python)................................................Passed
check yaml...............................................................Passed
trim trailing whitespace.................................................Passed
Validate GitHub Workflows................................................Passed
```

If you need to run it again on all files and not just staged files, run:

```
uv run pre-commit run --all-files
```

Local build

This application can be deployed as a 'frozen' application using pyinstaller with:

```
uv pip install -e .[freeze]
uv run pyinstaller frozen.spec
```

This generates application files at `dist/ansys_python_manager` and you can run it locally by executing `Ansys Python Manager.exe`.

Documentation

For building documentation, you can either run the usual rules provided in the Sphinx Makefile:

```
uv pip install -e .[doc]
uv run make -C doc/ html

# subsequently open the documentation with (under Linux):
<your_browser_name> doc/html/index.html
```

1.2 Architecture

The Electronic Transformer Toolkit (PyETK) main dependency is the PyAEDT Common Toolkit, a package that provides common methods and best practices for creating Ansys Electronics Desktop toolkits. To better understand the toolkit template architecture (backend and user interface) you can refer to the PyAEDT Common Toolkit Architecture. To understand how the Backend API is structured and see an example refer to Backend API reference. The UI API reference explains how the UI is built and its modules.

The PyETK's file structure is the following

```
ansys-pyetk
├──.github
│   └──workflows
├──.idea
│   └──inspectionProfiles
├──.reuse
```

(continues on next page)

(continued from previous page)

```
│   └──templates
├──doc
│   ├──changelog.d
│   ├──source
│   │   ├──examples
│   │   ├──getting_started
│   │   ├──toolkit
│   │   │   └──_autosummary
│   │   ├──_static
│   │   │   ├──assets
│   │   │   │   └──download
│   │   │   └──thumbnails
│   │   └──_templates
│   └──styles
│       └──config
│           └──vocabularies
│               └──ANSYS
├──installer
│   ├──assets
│   │   └──scripts
│   ├──hooks
│   └──linux
│       └──debian
├──src
│   └──ansys
│       └──aedt
│           └──toolkits
│               └──electronic_transformer
│                   ├──backend
│                   │   └──workflows
│                   ├──data
│                   └──ui
│                       └──windows
│                           ├──help
│                           ├──images
│                           └──transformer_input
└──tests
    └──backend
        ├──json_files
        ├──tests_backend_api
        └──tests_toolkit_rest_api
```

- .github: GitHub Action configuration.
- doc: Documentation structure.
- template: Toolkit code, split into backend and UI.
  - **backend:**
    Non-user-facing part of the toolkit for handling requests and preparing data for the UI. Key files include:
    * `rest_api.py`: Defines Flask entrypoints.
    * `api.py`: Defines the toolkit API.
    * `backend_properties.toml`: Defines common backend properties.

        * `models.py`: Defines the class for storing backend properties.
    - ui: UI part of the toolkit. Key files include:
        * `frontend_properties.toml`: Defines common UI properties.
        * `models.py`: Defines the class for storing UI properties.
- tests: Folder containing the backend unit tests.
- installer: Folder containing the installer configuration files.

CHAPTER

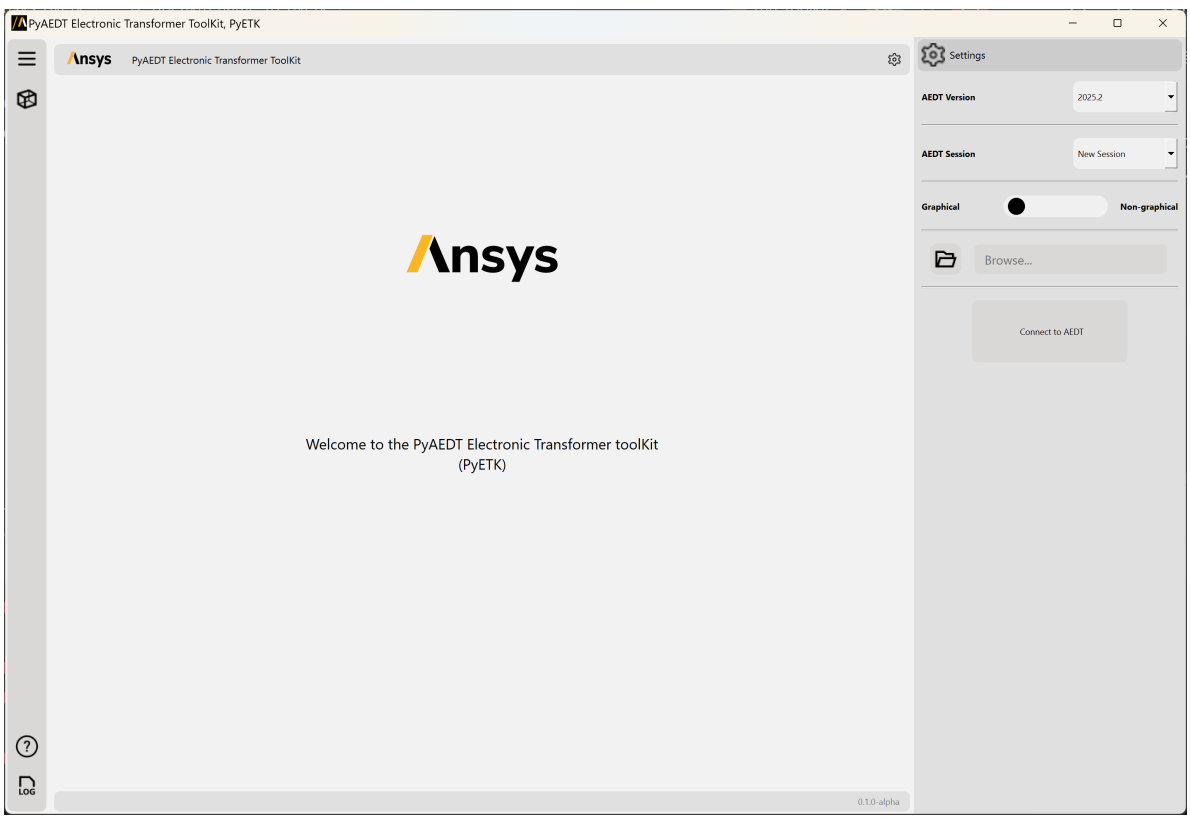
TWO

USER INTERFACE

This section describes how to use the PyETK. It assumes that you have already launched the toolkit. For toolkit installation, see *Installation*

1. On the **Settings** tab, specify settings for either creating an AEDT session or connecting to an existing AEDT session.

> **Note**
>
> If the PyETK Wizard is launched from AEDT, the **Settings** tab does not appear because the toolkit is directly connected to the specific AEDT session.



CHAPTER

THREE

API REFERENCE

The PyETK API contains the `ToolkitBackend` class, which provides methods for creating the whole model or some of the geometries, such as core, winding, bobbin, to help the user to visualize those parts only to create core types, bobbins, excitations, circuits, and ultimately put it all together as an electronic transformer model. You use the PyETK API at the toolkit level.

The common methods for creating an AEDT session or connecting to an existing AEDT session are provided by the Common toolkit library.

You can use the Toolkit API as shown in this example: ETK API example

class `ansys.aedt.toolkits.electronic_transformer.backend.api.`**`ToolkitBackend`**

Bases: `AEDTCommon`

Control the toolkit workflow.

This class provides methods to connect to a selected design and create the model or the parts of the transformer.

Examples

```
>>> from ansys.aedt.toolkits.electronic_transformer.backend.api import␣
↪ToolkitBackend
>>> toolkit_api = ToolkitBackend()
>>> toolkit_api.launch_aedt()
>>> toolkit_api.wait_to_be_idle()
>>> toolkit_api.create_core_geometry()
```

`create_bobbin_geometry`(*frontend_properties=None*)

Create the bobbin geometry.

Parameters

frontend_properties
[`dict`, `optional`] Frontend properties. The default is `None`.

Returns

`ansys.aedt.toolkits.electronic_transformer.backend.workflows.bobbin.Bobbin`
Bobbin object.

`create_core_geometry`(*frontend_properties=None*)

Create the core geometry.

Parameters

frontend_properties
[`dict`, `optional`] Frontend properties. The default is `None`.

Returns

ansys.aedt.toolkits.electronic_transformer.backend.workflows.core.Core
Core object.

create_model(*frontend_properties=None*)

Create the complete model.

Parameters

frontend_properties
[dict, optional] Frontend properties. The default is None.

Returns

ansys.aedt.toolkits.electronic_transformer.backend.workflows.etk.ETK
ETK object.

create_winding_geometry(*frontend_properties=None*)

Create the winding geometry.

Parameters

frontend_properties
[dict, optional] Frontend properties. The default is None.

Returns

ansys.aedt.toolkits.electronic_transformer.backend.workflows.winding.Winding
Winding object.

read_props_from_json(*file_path*)

Read properties from a JSON file.

Parameters

file_path
[str] Path to the JSON file.

validate_json(*data*)

Validate input data from a JSON file.

Parameters

data
[pathlib.Path] Path to the input data in JSON format.

CHAPTER

FOUR

EXAMPLES

Examples are available to demonstrate how to use pyETK in different ways.

Create a transformer model. Example of how to use the pyETK API to create the whole model.

ETK_example

CHAPTER

FIVE

RELEASE NOTES

This document contains the release notes for the project.

5.1 0.1.0 - January 30, 2026

Added

JSON Example Files Added	#5
Settings Group Added to LHS UI Column	#186
Add Support for Bespoke and Reference Core Dimensions	#196
update json format	#210
BE tests improvement	#214
UI Save Button	#215
Add Connections to Data Exchange	#231
Add FE Tests	#232

Dependencies

Bump actions/setup-python from 6.0.0 to 6.1.0	#212
Bump softprops/action-gh-release from 2.3.3 to 2.5.0	#217
Bump actions/checkout from 5.0.0 to 6.0.1	#218
Bump actions/download-artifact from 5.0.0 to 7.0.0	#223
Bump actions/upload-artifact from 4.6.2 to 6.0.0	#224

Documentation

Remove Fixed Issues from README.rst	#11
Correct README.rst syntax	#13
Fix Help Menu Title “PyETK Transformer Toolkit”	#18
Improved backend API documentation	#33
Update badge link in README.rst	#44

Fixed

Fixed pyaedt arguments	#22
Backend model json_version typo	#49
Update backend properties, one property at a time	#54
Fixed layer spacing	#60
Rename create_transformer to create_model	#188
Fix load in side 1	#191
Fixes BE issues	#197
Frequency unit conversion	#205
Read from and Write gui_properties	#216
import fe_properties as properties	#229

Maintenance

Update license to Apache-2.0	#2
Clean doc and tests jobs	#8
Refactor into multiple workflors	#21
Update project's root level files	#24
Add missing checkout and wheelhouse	#56
Add missing job dependency in release job	#64
update CHANGELOG for v0.1.0a0	#178
bump 0.2.dev0	#179

Miscellaneous

Update Save As button text	#45
Update main_page.py	#46
PostProcessing and new HTTP request through /create_model	#184
remove/modify PyAEDT from toolkit naming convention	#189
Frontend Data and DataBase Manager	#228
Connections Definitions and Minor FE Fixes	#230

Test

Add additional bobbin unit tests	#6
Add additional data_manager unit tests	#7
Add circuit test	#9
Improved core tests	#14
UI Core with and without Airgap	#15
Add tests to FE data manager	#16

CHAPTER

SIX

WHAT IS THIS LIBRARY?

PyETK leverages PyAEDT methods and best practices to develop electronic transformers. It brings numerous advantages like enhancing efficiency, consistency, and collaboration in the creation and development of AEDT magnetic component models.

PYTHON MODULE INDEX

a

ansys.aedt.toolkits.electronic_transformer.backend.api, 11

INDEX

A

ansys.aedt.toolkits.electronic_transformer.backend.api
module, 11

C

create_bobbin_geometry() (*ansys.aedt.toolkits.electronic_transformer.backend.api.ToolkitBackend method*), 11
create_core_geometry() (*ansys.aedt.toolkits.electronic_transformer.backend.api.ToolkitBackend method*), 11
create_model() (*ansys.aedt.toolkits.electronic_transformer.backend.api.ToolkitBackend method*), 12
create_winding_geometry() (*ansys.aedt.toolkits.electronic_transformer.backend.api.ToolkitBackend method*), 12

M

module
ansys.aedt.toolkits.electronic_transformer.backend.api, 11

R

read_props_from_json() (*ansys.aedt.toolkits.electronic_transformer.backend.api.ToolkitBackend method*), 12

T

ToolkitBackend (*class in ansys.aedt.toolkits.electronic_transformer.backend.api*), 11

V

validate_json() (*ansys.aedt.toolkits.electronic_transformer.backend.api.ToolkitBackend method*), 12